

Propuesta Técnica — Asistente de Ayuda con Claude

Estado: PROPUESTA. Nada de esto está implementado. Documento de diseño para revisión y autorización antes de cualquier desarrollo.

Autoría: auditoría técnica junio 2026. Basado en la arquitectura real del ERP (Next.js 16 + Supabase multi-tenant + Vercel).

1. Visión

Un **asistente de ayuda contextual** embebido en el ERP (botón flotante / panel lateral) que:

- Responde preguntas sobre módulos, pantallas, formularios y procesos.
- Sabe **en qué pantalla está el usuario** y qué módulos tiene su empresa.
- Puede **analizar capturas de pantalla** que el usuario sube (ej. un error).
- Usa la documentación propia (docs/assistant/*.md) y screenshots reales como base de conocimiento (RAG).
- Es **multi-tenant seguro**: jamás mezcla información entre empresas ni expone datos de otra.

Qué NO debe hacer (alcance v1)

- No ejecuta acciones en el ERP (no crea ventas, no factura, no modifica datos).
- No responde con datos operativos de otra empresa, ni siquiera agregados.
- No reemplaza al soporte humano: escala cuando no sabe.

2. Arquitectura propuesta (alto nivel)

ERP (Next.js / Vercel)

```
UI: <AssistantWidget /> (panel lateral, por módulo)
  | contexto: { pathname, módulo, rol, empresa_id, locale }
  | ▼
POST /api/assistant/chat (Route Handler, streaming SSE)
  | 1. resolveApiAuthContext() ← reutiliza el auth existente (NO se modifica)
  | 2. Guardrails de tenant (módulos habilitados, rate limit, log)
  | 3. Retrieval (RAG):
  |   | embedding de la pregunta → pgvector (tabla assistant_kb_chunks)
  |   | filtros: módulo actual (boost), módulos habilitados del tenant (hard)
  | 4. Prompt = system (rol+reglas) + contexto pantalla + chunks + historial
  | ▼
Claude API (Messages, streaming)
  | • texto: claude-haiku-4-5 (default) / claude-sonnet-4-6 (escalado)
  | • imágenes del usuario: vision en el mismo request
  | • prompt caching para el system prompt + doc base
```

Pipeline de ingesta (offline, manual/CI)

```
docs/assistant/*.md + screenshots → chunking → embeddings → pgvector
```

| (corpus GLOBAL del producto, versionado en git; sin datos de clientes) |

Principio rector: el conocimiento del asistente es **documentación del producto (global)**; el **contexto del usuario/tenant** solo se usa para *filtrar y personalizar*, nunca se indexa junto al corpus. Esto elimina por diseño el riesgo de mezclar datos entre clientes.

3. Respuestas a las 14 preguntas del análisis

3.1 Qué información necesita el asistente

1. **Corpus de producto (global, no sensible):** los 15 documentos de `docs/assistant/` + descripciones de screenshots + FAQ. Es idéntico para todos los tenants.
2. **Contexto de sesión (efímero, por request):** pathname/módulo actual, rol del usuario, módulos habilitados de la empresa, idioma.
3. **Adjuntos del usuario (efímeros):** capturas que el usuario sube para explicar su problema.
4. **(Fase 2, opcional) Datos del tenant en modo solo lectura acotada:** p. ej. "¿por qué esta factura está rechazada?" requeriría leer el evento SIFEN de ESA factura, siempre vía el cliente Supabase del propio usuario (RLS), nunca con service role.

3.2 Cómo obtener contexto de pantalla

- El widget envía `pathname` + parámetros no sensibles. El mapeo ya existe: `pathRequiresModuleSlug()` (`src/lib/modulos/route-slug-map.ts`) traduce `pathname` → módulo.
- Tabla estática `pantalla` → doc relevante (ej. `/crm` → `crm.md`) para *boost* de retrieval.
- Opcional v1.5: el widget puede capturar el screenshot del viewport (con consentimiento explícito del usuario, botón "adjuntar mi pantalla") y enviarlo a Claude vision.

3.3 Cómo obtener contexto del usuario

- Reutilizar `resolveApiAuthContext()` (ya existe, no se modifica): devuelve usuario, rol, `empresa_id`. El asistente personaliza el tono y filtra contenido por rol (ej. instrucciones de configuración solo si es admin).

3.4 Cómo obtener contexto del tenant

- `resolveEffectiveModules()` (ya existe) da los módulos habilitados → el retrieval **excluye** documentación de módulos que la empresa no tiene (evita "fantasmas": explicar pantallas que el usuario nunca podrá ver).
- `empresas.data_schema` no es necesario para v1 (no se tocan datos del tenant).

3.5 Cómo evitar mezcla de clientes

Defensa en capas:

1. **Por diseño:** el corpus RAG es global y sin datos de clientes. No hay nada "de otro cliente" que recuperar.
2. **Si en fase 2 se indexa contenido por tenant** (ej. notas internas de la empresa): columna `empresa_id NOT NULL` en la tabla de chunks + **RLS** con `puede_acceder_empresa()` (patrón ya existente) + filtro explícito en la query + tests automatizados de aislamiento.
3. **Sesiones de chat:** tabla `assistant_conversations` con `empresa_id` + RLS.
4. **Prompt:** el system prompt prohíbe revelar información de configuración de otras empresas; pero la seguridad NO depende del prompt (capas 1–3 son las reales).
5. **Logging y auditoría** de cada pregunta/respuesta por tenant.

3.6 Cómo implementar RAG

- **Vector store: pgvector en el mismo Postgres de Supabase** (extensión soportada nativamente). Evita un servicio externo nuevo, hereda backups y RLS.
- Esquema propuesto (nuevo, **no toca tablas existentes**):

```
-- schema propio del asistente, p. ej. assistant
create table assistant.kb_documents (
  id uuid primary key default gen_random_uuid(),
  slug text unique not null,          -- 'crm', 'facturas', ...
  module_slug text,                  -- módulo del ERP al que pertenece (null = transversal)
  title text not null,
  version text not null,             -- hash del archivo fuente en git
  updated_at timestamptz default now()
);

create table assistant.kb_chunks (
  id uuid primary key default gen_random_uuid(),
  document_id uuid references assistant.kb_documents(id) on delete cascade,
  heading text,                      -- sección (## ...) para citación
  content text not null,             -- 300-800 tokens por chunk
  screenshot_paths text[],           -- capturas asociadas al chunk
  embedding vector(1024),            -- voyage-3.5 / voyage-3.5-lite
  tsv tsvector                       -- búsqueda léxica (híbrida)
);
create index on assistant.kb_chunks using hnsu (embedding vector_cosine_ops);
```

- **Retrieval híbrido:** vector (cosine) + texto (`tsvector` , maneja bien términos exactos como "KuDE", "CDC", "timbrado") + boost por `module_slug` = módulo actual del usuario; filtro duro por módulos habilitados del tenant. Top-k ≈ 6-8 chunks.
- **Chunking:** por encabezado de sección (la estructura de los docs ya está pensada para esto: Objetivo / Flujos / FAQ / Errores). FAQ: un chunk por Q&A.
- **Embeddings:** Voyage AI (`voyage-3.5-lite` , multilingüe, recomendado por Anthropic) o el endpoint de embeddings disponible; el corpus es pequeño (~50-100 KB), el costo de indexación es despreciable y se reindexa completo en cada cambio de docs (CI).

3.7 Cómo almacenar documentación

- **Fuente de verdad: git** (`docs/assistant/*.md`) — versionada, revisable por PR, editable por humanos. Es lo que ya produce esta auditoría.
- **Ingesta:** script (`scripts/assistant-ingest.ts`) que corre en CI o manualmente: parsea los .md, chunkea, calcula embeddings y hace upsert por hash de versión. Idempotente.
- Los documentos describen la funcionalidad **base**; las variantes por tenant se resuelven con el filtro de módulos, no duplicando documentos.

3.8 Cómo almacenar screenshots

- **Supabase Storage**, bucket nuevo `assistant_kb` (público-lectura solo vía URL firmada o detrás del endpoint del asistente). Los archivos de `docs/assistant/screenshots/` se suben con el mismo script de ingesta, con paths estables (`<módulo>/<archivo>.png`).
- En la tabla de chunks se guardan los paths asociados → la respuesta del asistente puede incluir la captura ("así se ve la pantalla de colas") renderizada por el widget.
- **Regla de oro:** los screenshots del corpus se toman de un **tenant demo** (datos ficticios), nunca de empresas reales. Los actuales (empresa del tester con datos reales) sirven para diseño, pero antes de producción deben regenerarse desde un tenant demo o anonimizarse.

3.9 Cómo incorporar análisis de imágenes

Dos usos distintos:

1. **Usuario sube captura de su pantalla/error** → se envía como bloque `image` en el mismo request a Claude (vision nativa de los modelos Claude 4.x). El modelo identifica la pantalla, lee el mensaje de error y el RAG aporta el contexto del módulo. Sin infraestructura extra.
 - Límite: ~4 MB por imagen vía widget; redimensionar client-side a $\leq 1568\text{px}$ (óptimo de costo).
2. **Screenshots del corpus** → NO se envían como imagen en cada request (caro). En ingesta se genera **una descripción textual** de cada captura (one-shot con Claude vision) que se indexa como chunk; la imagen en sí solo se devuelve como adjunto en la respuesta.

3.10 Cómo mantener costos bajos

Técnica	Detalle
Modelo por defecto barato	<code>claude-haiku-4-5</code> para Q&A doc-grounded (excelente relación calidad/costo). Escalado a <code>claude-sonnet-4-6</code> solo si la pregunta incluye imagen compleja o el usuario pide más detalle
Prompt caching	El system prompt + reglas (~2–4 K tokens) se cachea (90 % de descuento en lectura de caché); con tráfico sostenido el costo por consulta cae drásticamente
RAG en vez de contexto gigante	Solo 6–8 chunks (~2–3 K tokens) por consulta, no todo el corpus
Historial acotado	Últimos ~6 turnos por conversación; resúmenes si se alarga
Rate limiting por tenant	N consultas/usuario/día según plan (tabla de cuotas); evita abuso
Cache de respuestas frecuentes	Hash(pregunta normalizada + módulo) → respuesta, TTL 24 h, para las FAQ repetidas
Descripciones de screenshots pre-computadas	Vision una sola vez en ingesta, no por consulta

Estimación (orden de magnitud, precios públicos jun-2026, Haiku 4.5 ≈ \$1/M in, \$5/M out):

- Consulta típica: ~3,5 K tokens entrada (mayoría cacheada) + ~400 tokens salida ≈ **\$0,003–0,005 por consulta**.
- 100 clientes × 20 consultas/mes = 2.000 consultas ≈ **\$6–10/mes** de API.
- Incluso con 10× de tráfico y escalado parcial a Sonnet: < \$150/mes. El costo dominante será desarrollo y curación de docs, no la API.

3.11 Cómo escalar a cientos de clientes

- **Stateless**: el endpoint del asistente es un Route Handler sin estado; escala con Vercel.
- **Corpus único global** → el tamaño del índice NO crece con la cantidad de clientes, solo con la cantidad de documentación. pgvector con HNSW maneja esto trivialmente (miles de chunks).
- **Cuotas por plan** (columna en `empresas` o tabla de límites) para gobernar el costo.
- **Self-hosted dedicado**: la instancia Neura dedicada (commit reciente) puede apuntar al mismo corpus global o a uno propio; el diseño por schema (`assistant.*`) lo permite sin cambios.
- Si en fase 2 hay contenido por tenant, el índice particiona naturalmente por `empresa_id` (filtro + RLS), con cardinalidad baja por tenant.

3.12 Riesgos técnicos

Riesgo	Mitigación
Documentación desactualizada → respuestas erróneas	Docs en git junto al código; checklist de PR ("¿cambió una pantalla? actualizar docs/assistant"); versión visible en cada respuesta

Riesgo	Mitigación
Alucinaciones (inventar features)	RAG estricto + instrucción "si no está en la documentación, decí que no sabés y ofrecé escalar a soporte"; citar la fuente (doc + sección)
Latencia (RAG + LLM)	Streaming SSE (el usuario ve la respuesta crecer); Haiku es rápido; retrieval en Postgres local <50 ms
Acoplamiento al deploy del ERP	El asistente es aditivo: widget + 1 endpoint + schema nuevo. Si falla, el ERP no se afecta (feature flag por empresa, módulo <code>asistente</code> en el catálogo de módulos existente)
Límites de rate de la API Claude	Reintentos con backoff, colas de degradación ("alto tráfico, reintentá"), cuotas por tenant
Crecimiento del historial de chats	TTL/archivado de conversaciones (ej. 90 días)

3.13 Riesgos de seguridad

Riesgo	Mitigación
Fuga entre tenants	Corpus global sin datos de clientes (por diseño); RLS en toda tabla nueva con <code>empresa_id</code> ; tests de aislamiento automatizados
Prompt injection (usuario o texto en imagen intenta manipular al asistente)	El asistente no tiene herramientas de escritura en v1 (no hay nada que inyectar contra); system prompt endurecido; nunca incluir secretos en el prompt
Datos sensibles en imágenes subidas	Aviso al usuario; no persistir imágenes por defecto (efímeras al request); si se guardan para soporte, bucket privado por tenant + TTL
Exposición de configuración interna	El corpus se cura: no incluir tokens, URLs internas, detalles de infraestructura (esta auditoría ya siguió esa regla)
Abuso / costos	Auth obligatoria (sesión Supabase existente), rate limit por usuario y tenant, logging
PII en logs de conversación	Minimizar retención, anonimizar teléfono/RUC en analytics

3.14 Plan de implementación recomendado

Fase 0 — (esta auditoría) ☒ Documentación funcional + screenshots + mapa del sistema + esta propuesta.

Fase 1 — MVP doc-grounded (1–2 semanas de dev)

- Schema `assistant` (kb_documents, kb_chunks, conversations, messages) + RLS. (*Migración nueva, no toca nada existente — requiere autorización para aplicarla.*)
- Script de ingesta (md → chunks → embeddings → pgvector) corrido manualmente.
- Endpoint `POST /api/assistant/chat` (streaming) con retrieval híbrido + Haiku 4.5 + caching.
- Widget flotante (lateral) con contexto de pathname; rollout con feature flag a 1–2 empresas piloto (módulo `asistente` en el catálogo de módulos).
- Telemetría: pregunta, módulo, chunks usados, feedback 👍 / 🗣️.

Fase 2 — Contexto enriquecido (2–3 semanas) 6. Vision para capturas subidas por el usuario. 7. Tenant demo para regenerar screenshots "limpios" del corpus; respuestas con imagen. 8. Botón "adjuntar mi pantalla" (consentimiento explícito). 9. Cache de FAQ + cuotas por plan + dashboard interno de uso.

Fase 3 — Asistente contextual a datos (a evaluar) 10. Lectura acotada de datos del propio tenant (estado de UNA factura SIFEN, una conversación) SIEMPRE vía el cliente RLS del usuario; tool-use de Claude con herramientas read-only whitelisted. Requiere revisión de seguridad dedicada. 11. Acciones guiadas ("te llevo a la pantalla") → deep links, nunca mutaciones.

Criterios de éxito del MVP: ≥70 % de respuestas con feedback positivo; deflection de tickets de soporte; cero incidentes de aislamiento; costo API < \$0,01/consulta.

4. Flujo de una consulta (detalle)

Usuario (en /crm) pregunta: "¿cómo convierto un prospecto en cliente?"

1. Widget → POST /api/assistant/chat { message, pathname:"/crm", conversationId }

2. Backend: auth (sesión existente) → empresa, rol, módulos=[crm, clientes, ...]

3. Embedding de la pregunta (voyage-3.5-lite)

4. SQL: top-8 chunks WHERE module_slug IN (módulos del tenant) ORDER BY (similitud × boost si module_slug='crm')

5. Prompt:

system = rol del asistente + reglas + glosario ERP [cacheado]

context = "El usuario está en CRM Funnel. Rol: vendedor. Módulos: ..."

docs = chunks recuperados (crm.md → 'Convertir en cliente', faq, clientes.md)

user = pregunta (+ imagen si adjuntó)

6. Claude Haiku 4.5 (stream) → respuesta con pasos + cita "  CRM Funnel > Convertir en cliente" + screenshot opcional (crm/01-pipeline.png)

7. Persistir turno en assistant.messages (empresa_id, RLS) + métricas

5. Resumen de decisiones

Decisión	Elección	Por qué
Vector DB	pgvector (Supabase)	Cero infra nueva, RLS, backups existentes
Modelo	Haiku 4.5 default, Sonnet 4.6 escalado	Costo/latencia vs. calidad
Corpus	Markdown en git, global, sin datos de clientes	Versionado, seguro por diseño
Screenshots	Storage + descripciones pre-computadas	Vision una sola vez, no por consulta
Aislamiento	Corpus global + RLS en tablas nuevas + cuotas	Defensa en capas, no depende del prompt
Integración	1 endpoint + widget + schema nuevo	Aditivo: riesgo cero sobre lo existente
Multimodal	Vision nativa de Claude en el request	Sin OCR/infra adicional